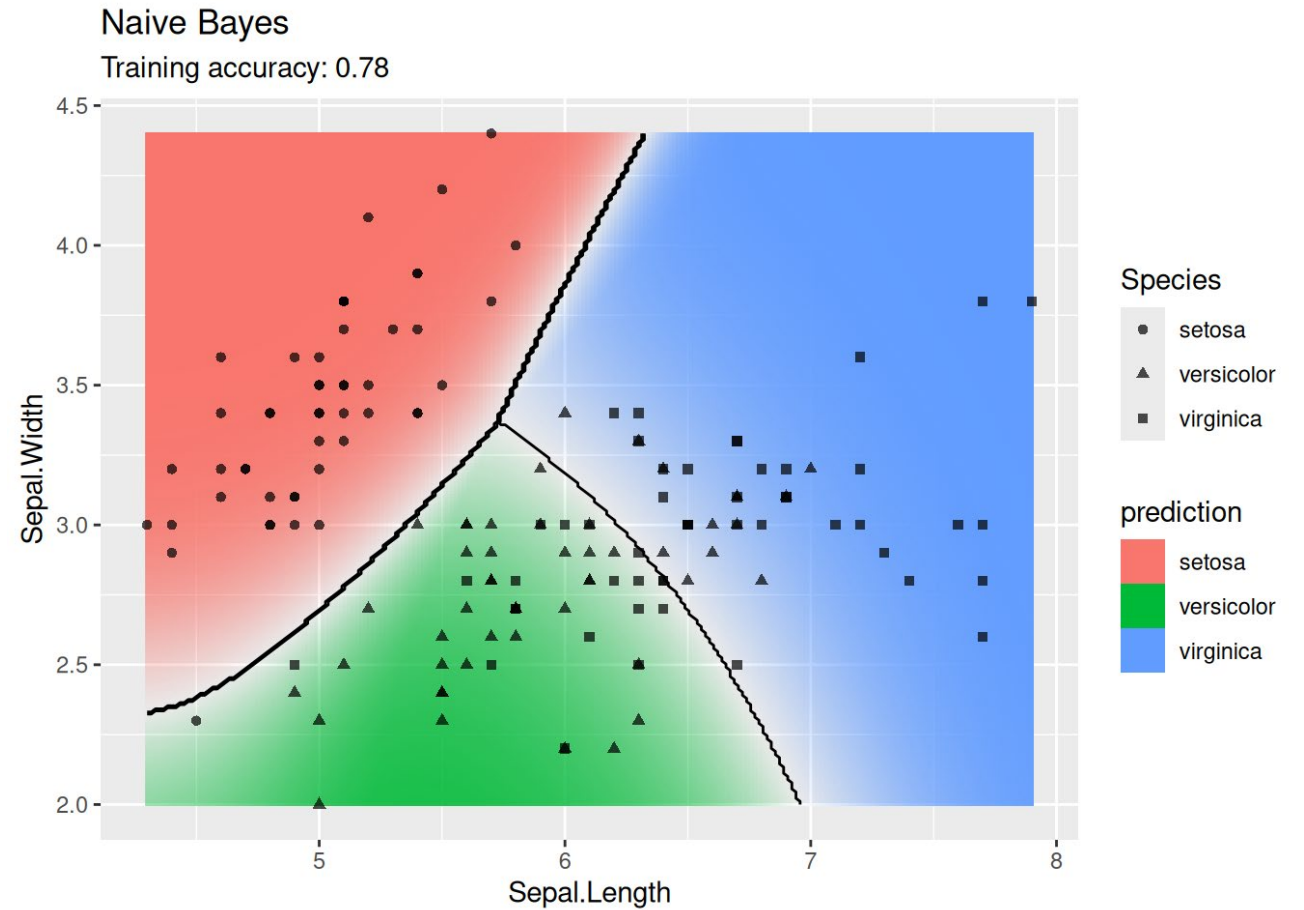


# Learning from Examples: Supervised Machine Learning



# Topics



# Agents and ML

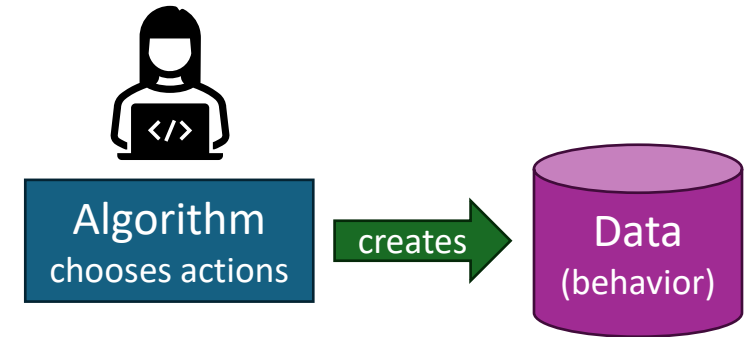


DeepAi.org with prompt: "A happy cartoon robot with an artificial neural network for a brain on white background learning to play chess"

# Learning from Examples

## Up until now in this course:

- **Hand-craft algorithms** to make rational/optimal or at least good decisions.  
Examples: Search strategies, heuristics, and constructing Bayesian networks.

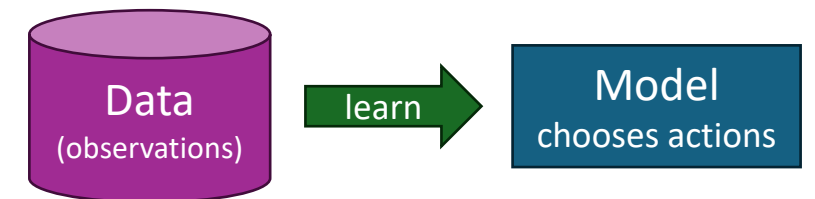


## Issues

- We may not be able to anticipate all possible future scenarios.
- We may have examples, but we do not know how to implement a solution.

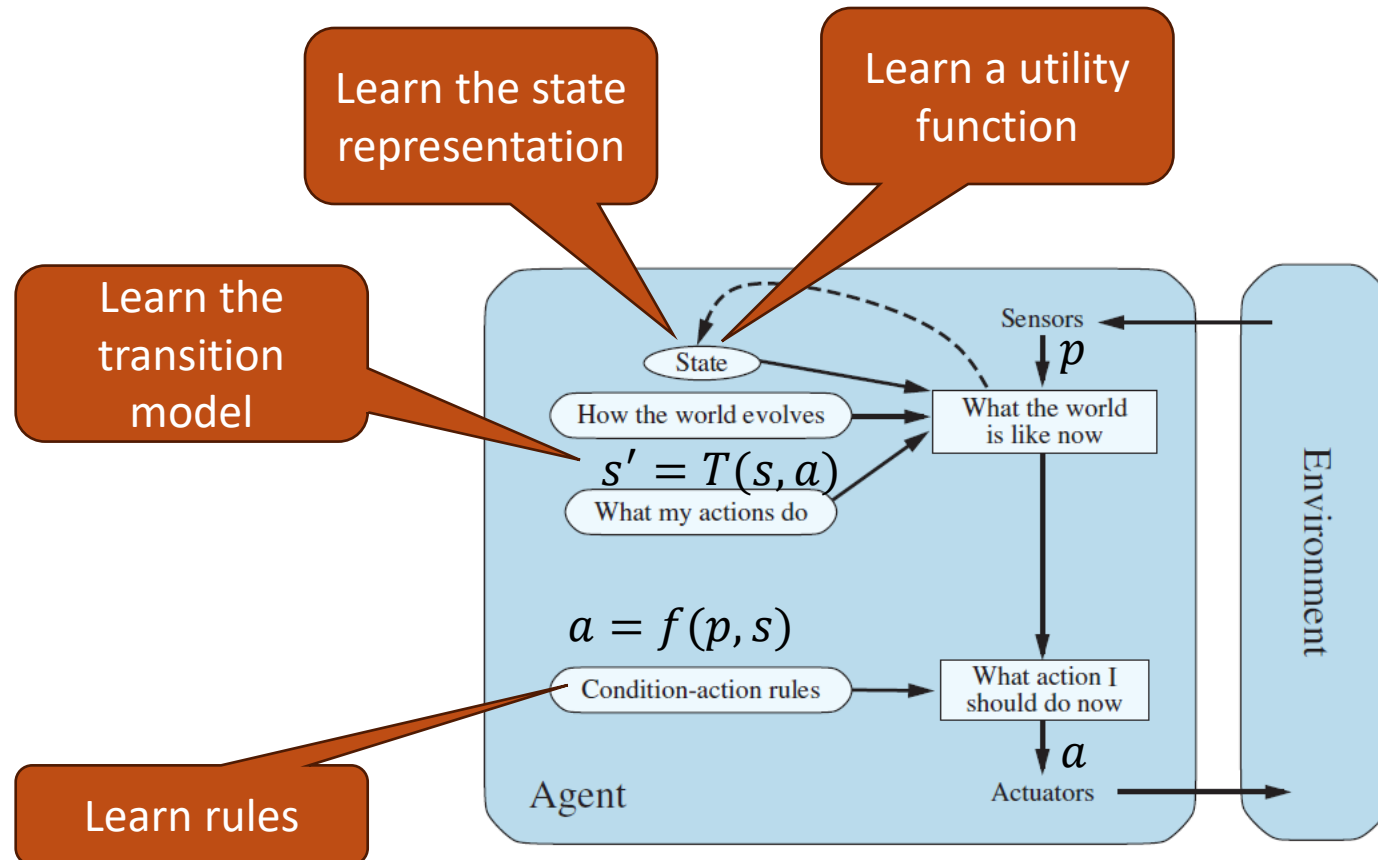
## Supervised Machine Learning

- Uses observations: training data with the correct answers.
- Learn a function (model) to map an input (e.g., state) to an output (e.g., action) representing the desired behavior.
- Examples:
  - Use a naïve Bayesian classifier to distinguish between spam/non-spam.
  - Learn a playout policy to simulate games (current board -> good move)



# Learning Components of an Agent

- We can learn many different components of an agent from examples
- **Example:** Learning components of a model-based reflex agent

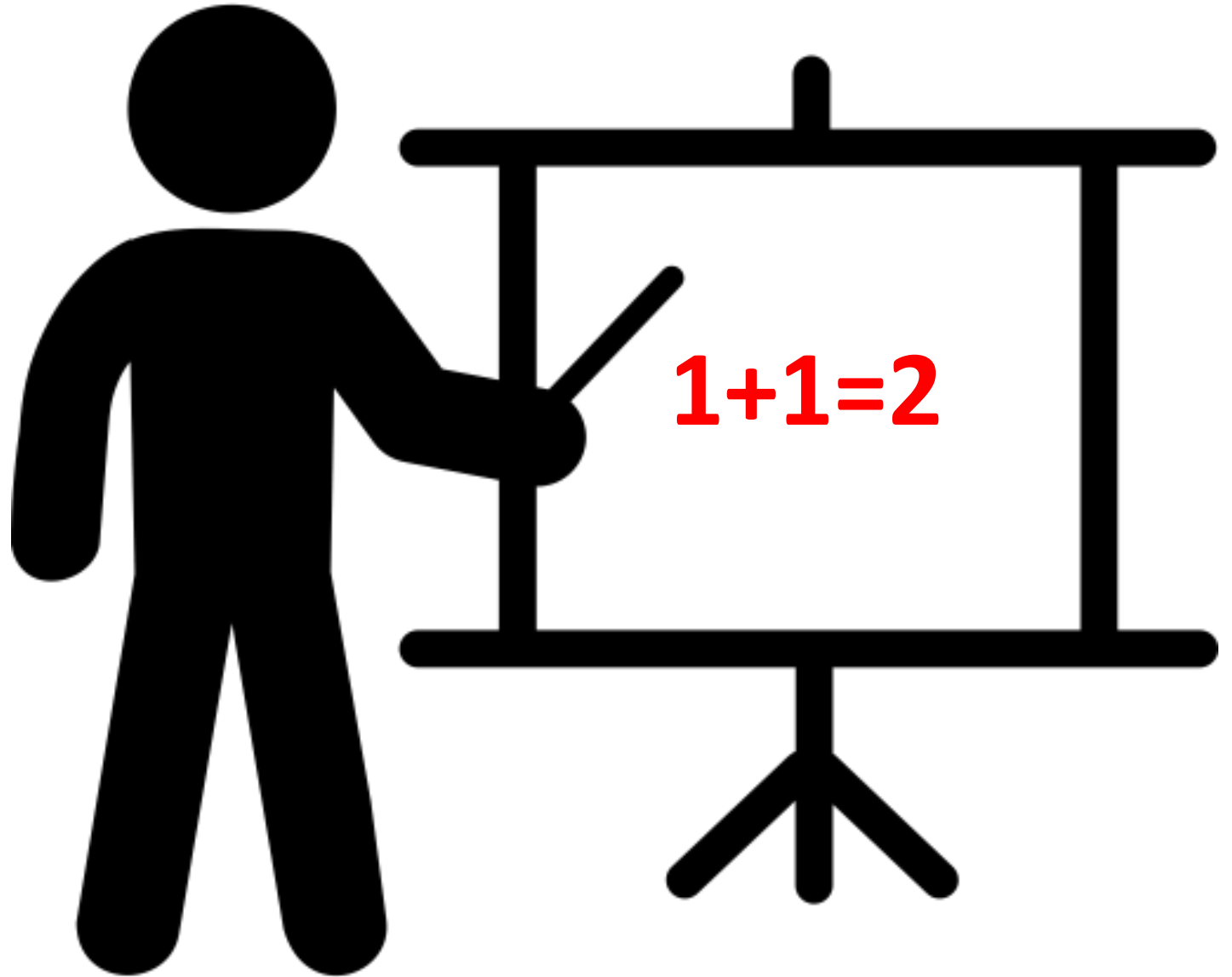




# Supervised Learning

---

Teaching a model to answer correctly



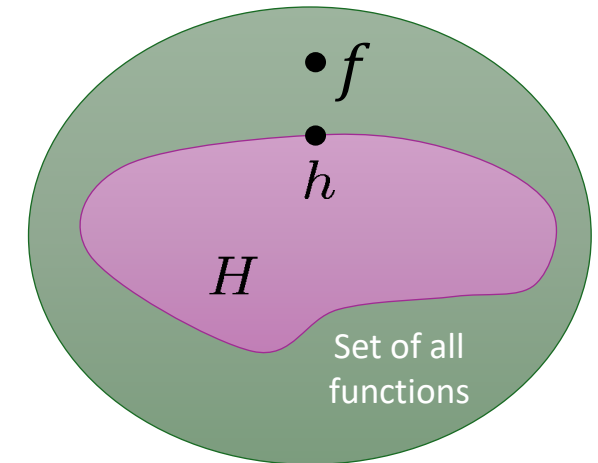
# Supervised Learning As Function Approximation

## Examples

- We assume there exists an unknown target function  $y = f(\mathbf{x})$  that produces iid (independent and identically distributed) examples, possibly with noise and errors.
- Examples are observed input-output pairs  $E = (\mathbf{x}_1, y_1), \dots, (\mathbf{x}_i, y_i), \dots, (\mathbf{x}_N, y_N)$ , where  $\mathbf{x}_i$  is a vectors called the feature vector.

## Learning problem

- Given a hypothesis space  $H$  of representable models.
- Find a hypothesis  $h \in H$  such that  $\hat{y}_i = h(\mathbf{x}_i) \approx y_i \forall i$
- That is, we want to approximate  $f$  by  $h$  using  $E$ .



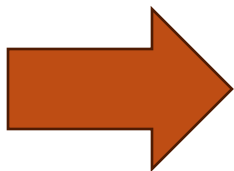
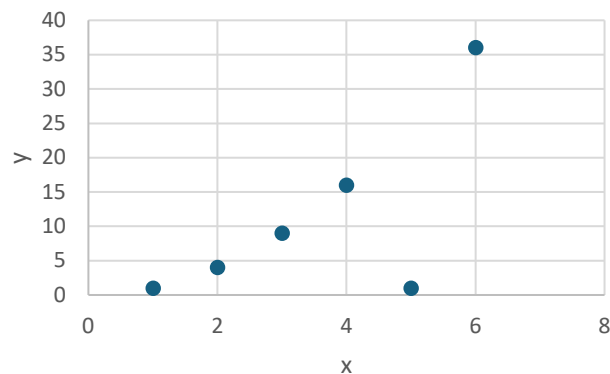
## Supervised learning includes

- Classification:  $y$  is a class labels. E.g.,  $\mathbf{x}$  are percepts and  $y$  is the chosen action.
- Regression:  $y$  is a real number. E.g.,  $\mathbf{x}$  are state features and  $y$  is the state utility.

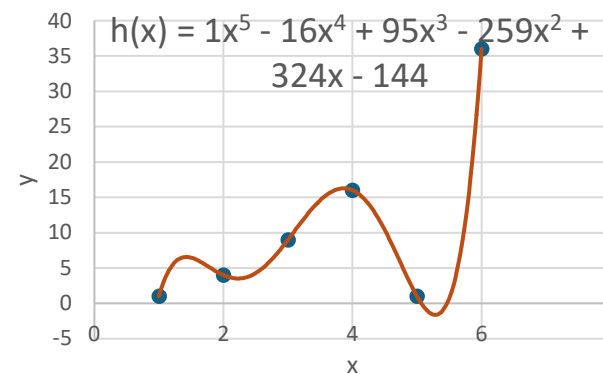
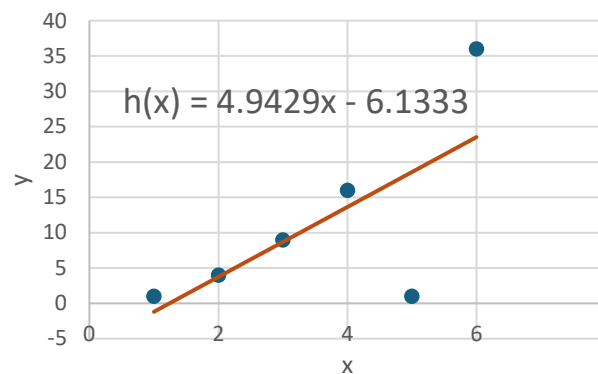
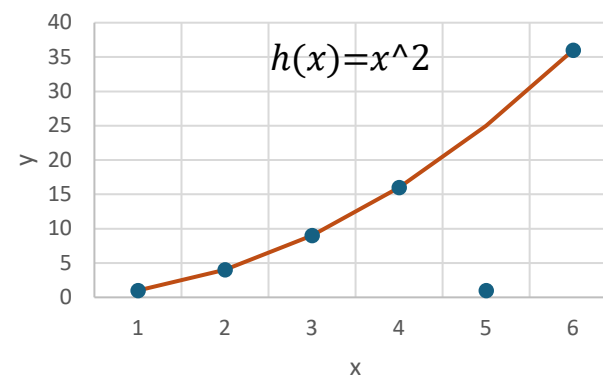
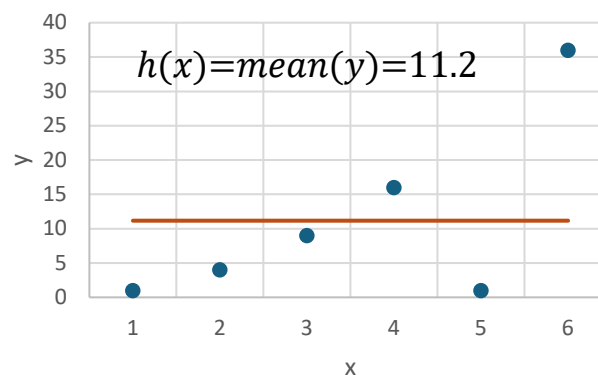
# Consistency vs. Simplicity

Example: Univariate curve fitting (regression, function approximation)

Examples  $f(x)$



Learned Models



**Consistency:**  $h(x_i) \approx y_i$  (minimize the error)

**Simplicity:** small number of model parameters.

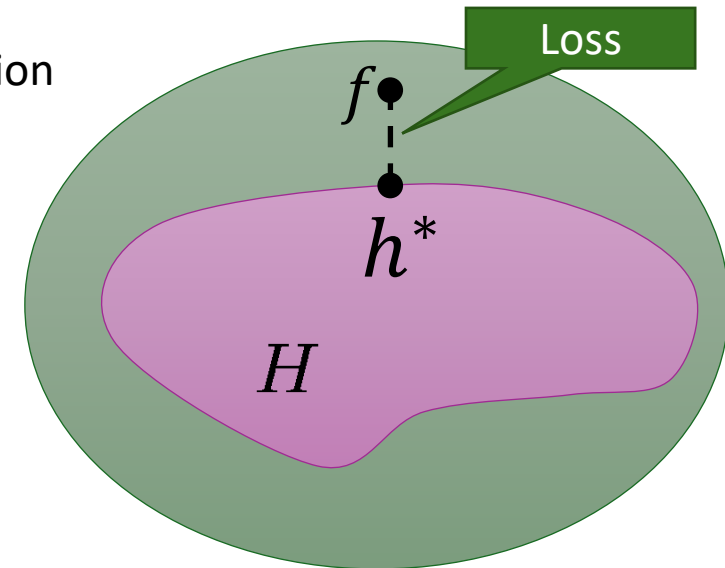
# Measuring Consistency using Loss

**Goal of learning:** Find a hypothesis that makes predictions that are consistent with the examples  $E = (\mathbf{x}_1, y_1), \dots, (\mathbf{x}_i, y_i), \dots, (\mathbf{x}_N, y_N)$ .

That is,  $\hat{y} = h(\mathbf{x}) \approx y$ .

**Measure mistakes:** Loss function  $L(y, \hat{y}) = L(f(\mathbf{x}), h(\mathbf{x}))$

- Absolute-value loss  $L_1(y, \hat{y}) = |y - \hat{y}|$
  - Squared-error loss  $L_2(y, \hat{y}) = (y - \hat{y})^2$
  - 0/1 loss  $L_{0/1}(y, \hat{y}) = 0$  if  $y = \hat{y}$ , else 1
  - Cross-entropy loss and many others...
- For Regression
- For Classification



# Learning Consistent Approximations

- Empirical loss

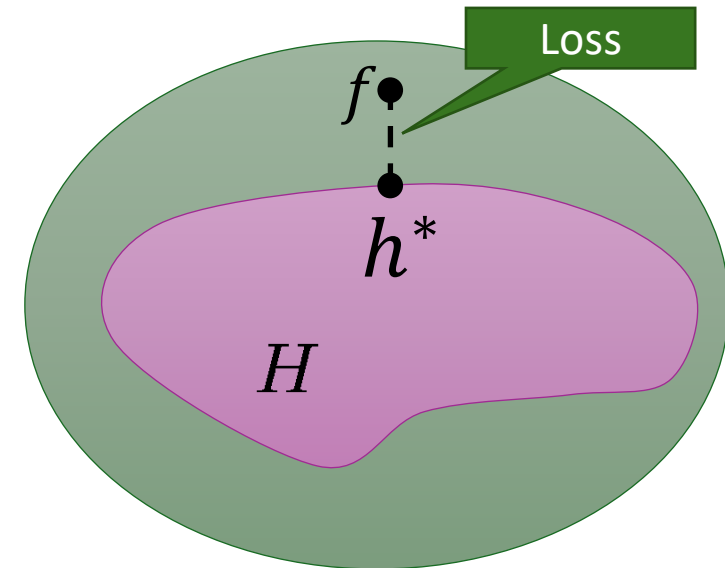
$$EmpLoss_{L,E}(h) = \frac{1}{|E|} \sum_{(x,y) \in E} L(y, h(x))$$

- Find the best hypothesis that minimizes the loss

$$h^* = \operatorname{argmin}_{h \in H} EmpLoss_{L,E}(h)$$

- Reasons for  $h^* \neq f$

- a) Realizability:  $f \notin H$
- b)  $f$  is nondeterministic.
- c) It is computationally intractable to search all of  $H$ , so we use a non-optimal heuristic like local search.



# The Most Consistent Classifier

**For 0/1 loss**, the empirical loss is minimized by the model that predicts for each  $\mathbf{x}$  the most likely class  $y$  using MAP (Maximum a posteriori) estimates. This is called the Bayes classifier.

$$h^*(\mathbf{x}) = \operatorname{argmax}_y P(Y = y \mid \mathbf{X} = \mathbf{x}) = \operatorname{argmax}_y \frac{P(\mathbf{x} \mid y) P(y)}{P(\mathbf{x})} = \operatorname{argmax}_y P(\mathbf{x} \mid y) P(y)$$

**Optimality:** The **Bayes classifier is optimal for 0/1 loss**. It is the most consistent classifier possible with the lowest possible error called the **Bayes error rate**. No better classifier exists for 0/1 loss!

**Issue:** The classifier requires to learn  $P(\mathbf{x} \mid y) P(y) = P(\mathbf{x}, y)$  from the examples.

- It **needs the complete joint probability** which requires in the general case a probability table with one entry for each possible feature combination in vector  $\mathbf{x}$ .
- This is impractical (unless a simple Bayesian network exists).  
Most classifiers try to approximate the Bayes classifier using a **simpler model** with fewer parameters.

# Simplicity

## Ease of use

- Simpler hypotheses have fewer model parameters to estimate and store. Also makes prediction faster.

## Generalization: How well does the hypothesis perform on new data?

- We do not want the model to be too specific to the training examples (an issue called **overfitting**).
- Simpler models typically generalize better to new examples.

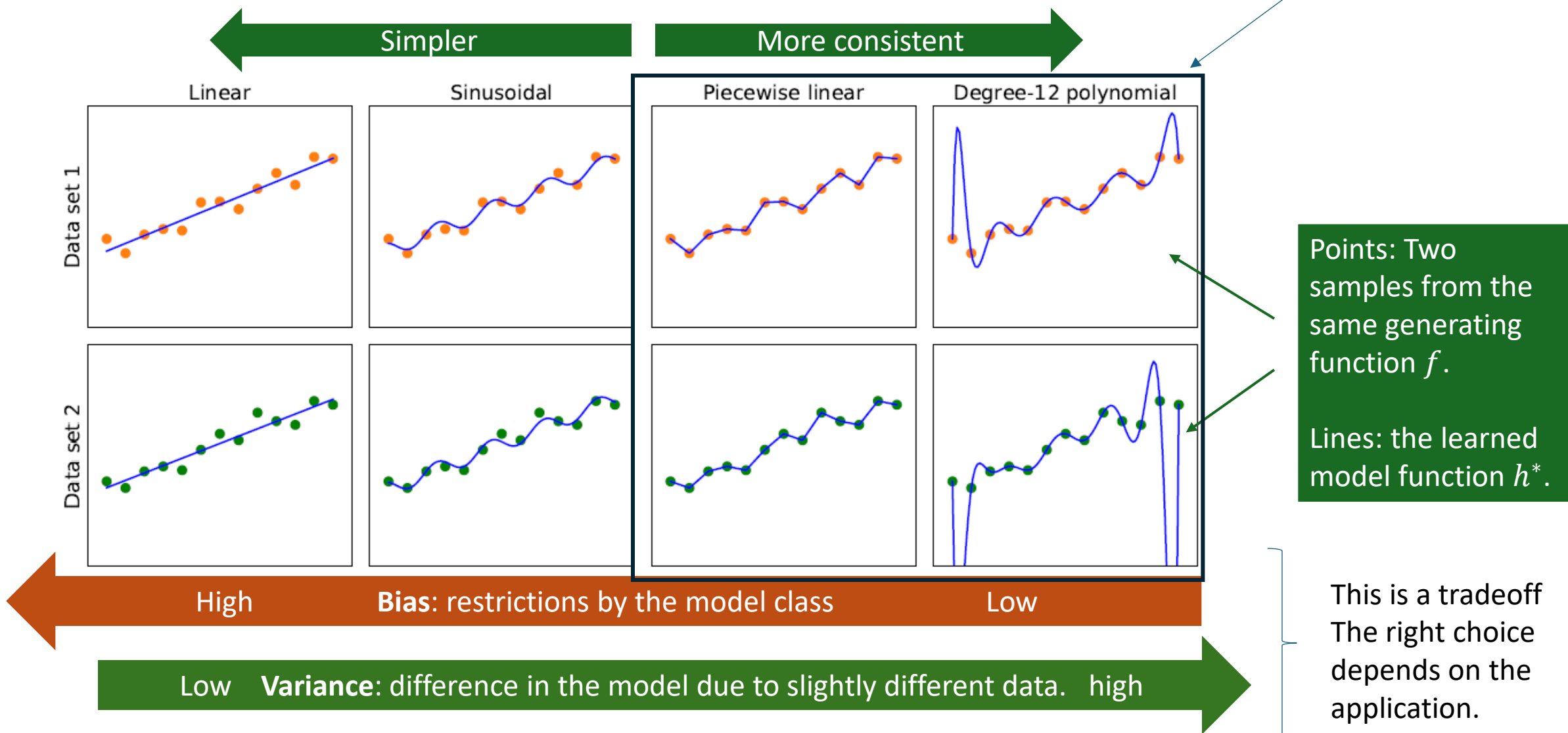
## How to achieve simplicity?

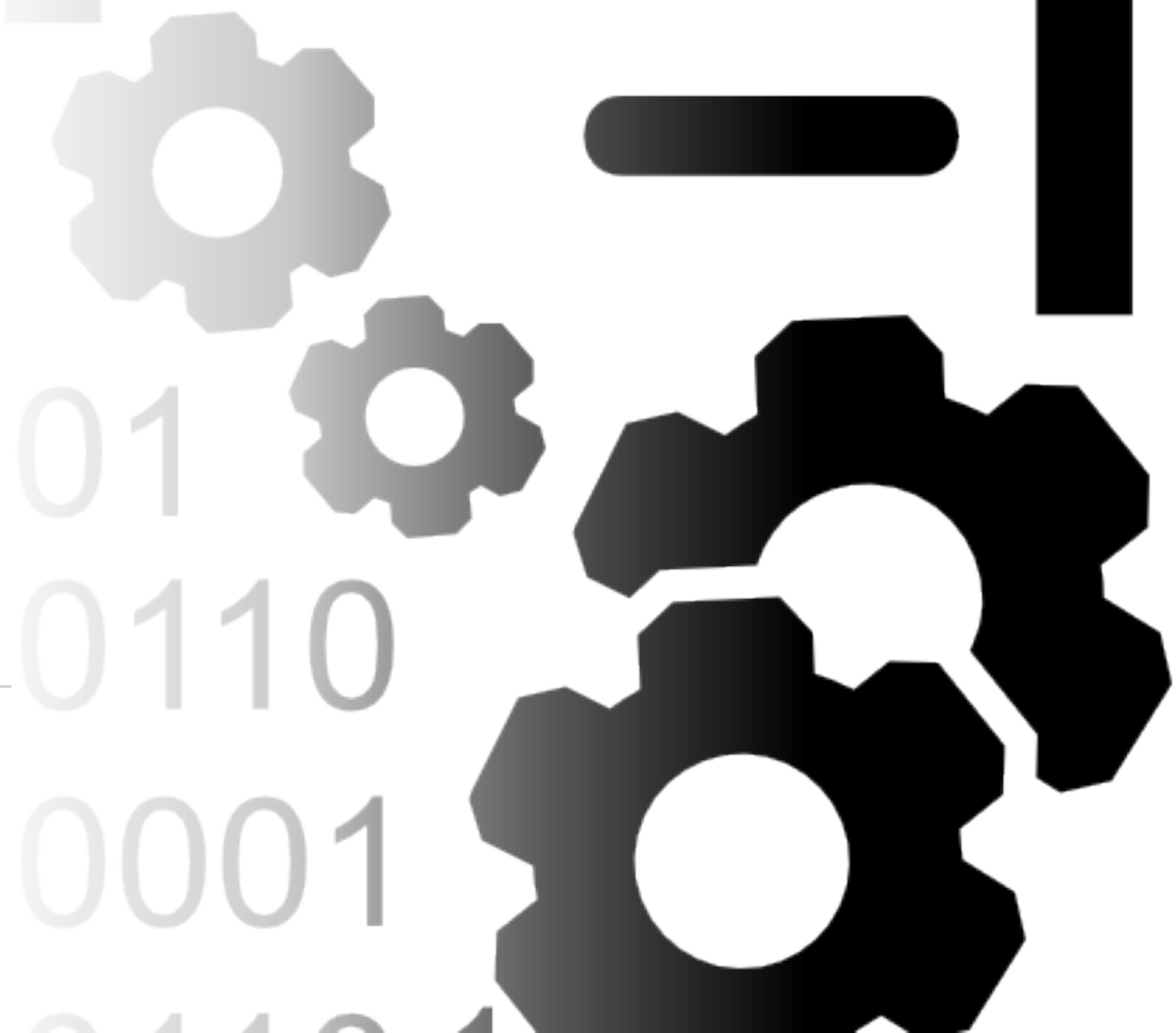
- Model bias:** Restrict  $H$  to simpler models (e.g., assumptions like independence, or only consider linear models).
- Feature selection:** use fewer variables from the feature vector  $\mathbf{x}$ .
- Regularization:** directly penalize the model for its complexity (e.g., number of parameters)

$$h^* = \operatorname{argmin}_{h \in H} \left[ \operatorname{EmpLoss}_{L,E}(h) + \underbrace{\lambda \operatorname{Complexity}(h)}_{\text{Penalty term}} \right]$$

Overfitting

# Model Selection: Bias vs. Variance





# Training Data

---

# The Dataset

Feature vector  $x$   
(Features, Variables, Attributes)

Class  
Label  $y$

Examples  
(Instances,  
Observation)

| Example  | Input Attributes |            |            |            |             |               |             |            |                |               | Output                |
|----------|------------------|------------|------------|------------|-------------|---------------|-------------|------------|----------------|---------------|-----------------------|
|          | <i>Alt</i>       | <i>Bar</i> | <i>Fri</i> | <i>Hun</i> | <i>Pat</i>  | <i>Price</i>  | <i>Rain</i> | <i>Res</i> | <i>Type</i>    | <i>Est</i>    | <i>WillWait</i>       |
| $x_1$    | <i>Yes</i>       | <i>No</i>  | <i>No</i>  | <i>Yes</i> | <i>Some</i> | <i>\$\$\$</i> | <i>No</i>   | <i>Yes</i> | <i>French</i>  | <i>0-10</i>   | $y_1 = \text{Yes}$    |
| $x_2$    | <i>Yes</i>       | <i>No</i>  | <i>No</i>  | <i>Yes</i> | <i>Full</i> | <i>\$</i>     | <i>No</i>   | <i>No</i>  | <i>Thai</i>    | <i>30-60</i>  | $y_2 = \text{No}$     |
| $x_3$    | <i>No</i>        | <i>Yes</i> | <i>No</i>  | <i>No</i>  | <i>Some</i> | <i>\$</i>     | <i>No</i>   | <i>No</i>  | <i>Burger</i>  | <i>0-10</i>   | $y_3 = \text{Yes}$    |
| $x_4$    | <i>Yes</i>       | <i>No</i>  | <i>Yes</i> | <i>Yes</i> | <i>Full</i> | <i>\$</i>     | <i>Yes</i>  | <i>No</i>  | <i>Thai</i>    | <i>10-30</i>  | $y_4 = \text{Yes}$    |
| $x_5$    | <i>Yes</i>       | <i>No</i>  | <i>Yes</i> | <i>No</i>  | <i>Full</i> | <i>\$\$\$</i> | <i>No</i>   | <i>Yes</i> | <i>French</i>  | <i>&gt;60</i> | $y_5 = \text{No}$     |
| $x_6$    | <i>No</i>        | <i>Yes</i> | <i>No</i>  | <i>Yes</i> | <i>Some</i> | <i>\$\$</i>   | <i>Yes</i>  | <i>Yes</i> | <i>Italian</i> | <i>0-10</i>   | $y_6 = \text{Yes}$    |
| $x_7$    | <i>No</i>        | <i>Yes</i> | <i>No</i>  | <i>No</i>  | <i>None</i> | <i>\$</i>     | <i>Yes</i>  | <i>No</i>  | <i>Burger</i>  | <i>0-10</i>   | $y_7 = \text{No}$     |
| $x_8$    | <i>No</i>        | <i>No</i>  | <i>No</i>  | <i>Yes</i> | <i>Some</i> | <i>\$\$</i>   | <i>Yes</i>  | <i>Yes</i> | <i>Thai</i>    | <i>0-10</i>   | $y_8 = \text{Yes}$    |
| $x_9$    | <i>No</i>        | <i>Yes</i> | <i>Yes</i> | <i>No</i>  | <i>Full</i> | <i>\$</i>     | <i>Yes</i>  | <i>No</i>  | <i>Burger</i>  | <i>&gt;60</i> | $y_9 = \text{No}$     |
| $x_{10}$ | <i>Yes</i>       | <i>Yes</i> | <i>Yes</i> | <i>Yes</i> | <i>Full</i> | <i>\$\$\$</i> | <i>No</i>   | <i>Yes</i> | <i>Italian</i> | <i>10-30</i>  | $y_{10} = \text{No}$  |
| $x_{11}$ | <i>No</i>        | <i>No</i>  | <i>No</i>  | <i>No</i>  | <i>None</i> | <i>\$</i>     | <i>No</i>   | <i>No</i>  | <i>Thai</i>    | <i>0-10</i>   | $y_{11} = \text{No}$  |
| $x_{12}$ | <i>Yes</i>       | <i>Yes</i> | <i>Yes</i> | <i>Yes</i> | <i>Full</i> | <i>\$</i>     | <i>No</i>   | <i>No</i>  | <i>Burger</i>  | <i>30-60</i>  | $y_{12} = \text{Yes}$ |

Alternative

Hungry  
Patrons

Reservation

Wait time

Task: Find a hypothesis (called “model”) to predict the class given the features.



# Feature Engineering

---

- Add more information sources as new variables to the model.
- Add **derived features** that help the classifier (e.g.,  $x_1x_2$ ,  $x_1^2$ ,  $\ln(x_1)$ ).
- **Embedding**: E.g., convert words to vectors where vector similarity between vectors reflects semantic similarity.
- **Feature Selection**: Which features should be used in the model is a model selection problem (choose between models with different features).
- (Deep) neural networks can perform “automatic” feature engineering called **end-to-end machine learning**.
- Example for Spam detection: In addition to words, add features for:
  - Have you emailed the sender before?
  - Have 1,000+ other people received the same email?
  - Is the email in ALL CAPS?

An abstract digital graphic on the left side of the slide. It features a dark blue background with several glowing blue cubes and rectangular frames. Binary code (0s and 1s) is visible within these shapes. Bright blue, green, and red light beams or points are scattered throughout the composition, creating a sense of depth and digital connectivity.

# Training Data in AI

---

- Training Data in AI can come from many sources
  - **Existing Data:** Download documents from the internet to train Large Language Models.
  - **Observation:** Record video of a task being performed (e.g., for self-driving cars).
  - **Simulation:** E.g., simulated games using a playout strategy.
  - **Expert feedback** on how well a task was performed.



# Training and Testing

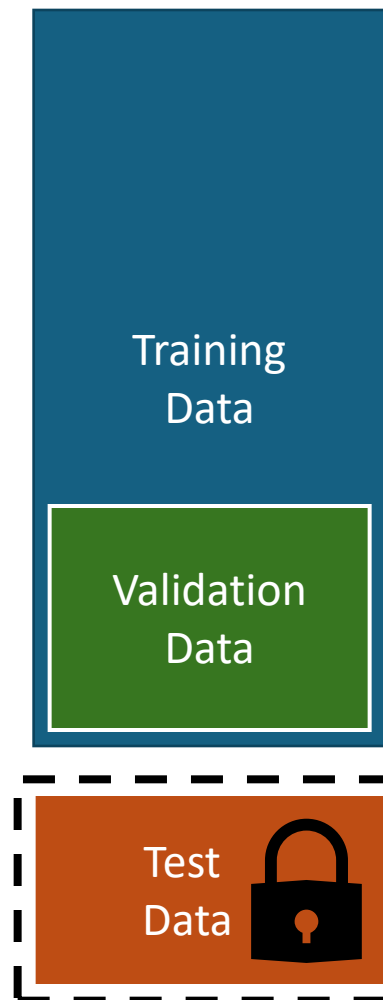


# Training a Model

1. Hold a **test set** back to estimate the generalization error (often 20%).
2. Hold a **validation data** set back from the training data (often 20%).
3. **Learn different models** using the training set with different **hyperparameters** (learning rate, regularization  $\lambda$ , maximal decision tree depth, selected features,... ). Often, a grid of possible hyperparameter combinations or some greedy search is used.
4. **Evaluate the models** using the validation data and choose the model with the best accuracy. Selecting the right type of model, hyperparameters, and features is called **model selection/hyper parameter tuning**.
5. **Learn the final model** with the chosen hyperparameters using all training (including validation data).

- Notes:

- The validation set was not used for training with different hyperparameters, so we get an estimate of the generalization error for comparing different hyperparameter settings.
- If no model selection is necessary, then no validation set is used.



# Testing: Evaluating the Final Model

The final model was trained on the training examples  $E$ . We want to test how well the model will perform on new examples  $T$  (i.e., how well it **generalizes to new data**). We use the held-back test data.

- **Testing loss**: Calculate the empirical loss for predictions on a testing data set  $T$  that is different from the data used for training.

$$EmpLoss_{L,T}(h) = \frac{1}{|T|} \sum_{(x,y) \in T} L(y, h(x))$$

- For classification we often use the **accuracy** measure, the proportion of correctly classified test examples.

$$accuracy(h, T) = \frac{1}{|T|} \sum_{(x,y) \in T} [h(x) = y] = 1 - EmpLoss_{L_{0/1},T}(h)$$

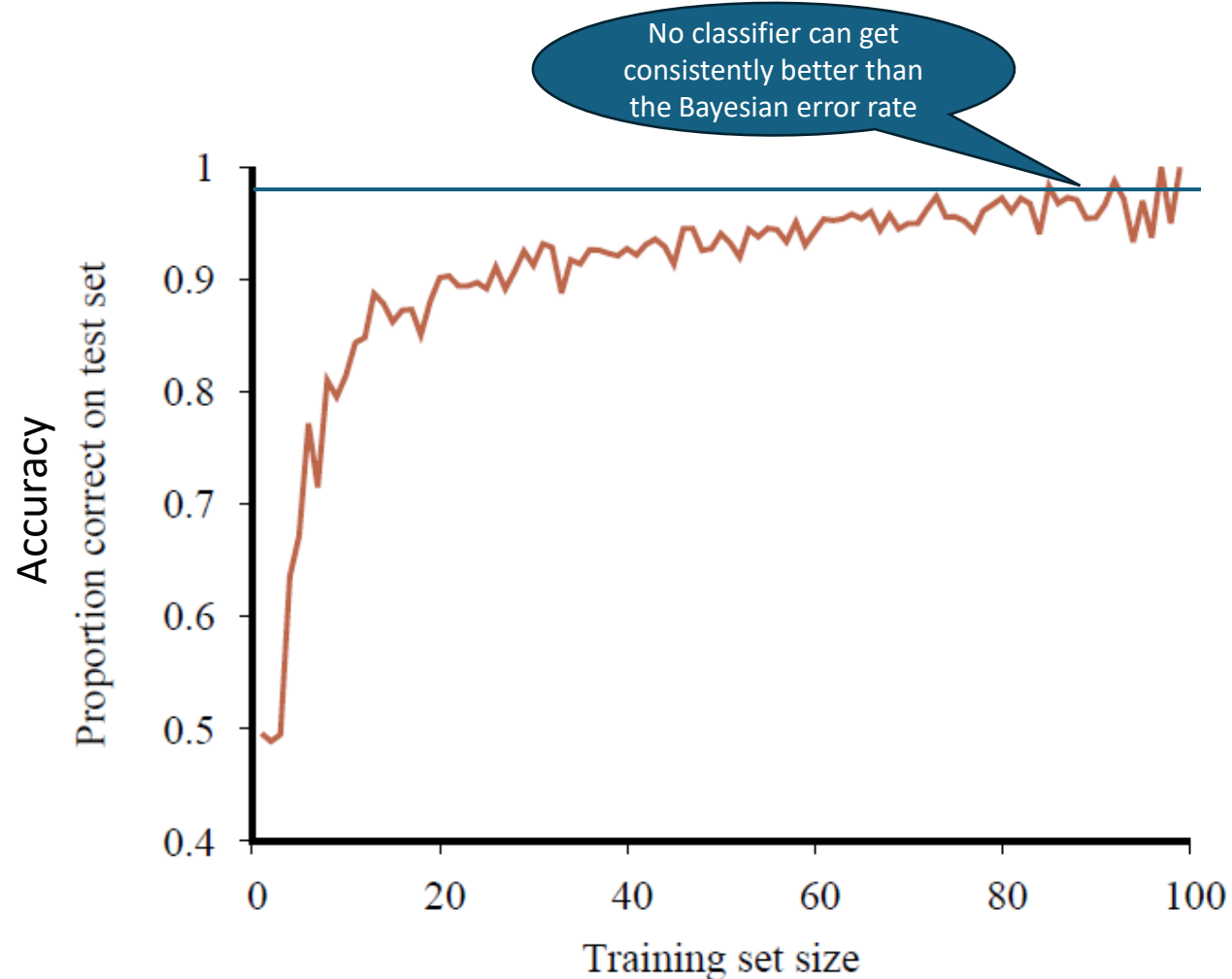
$[c]$  is an indicator function returning 1 if  $c = True$  and otherwise 0

Training  
Data  
 $E$

Test  
Data  $T$



# Learning Curve: The Effect the Training Data Size



Accuracy increases when the amount of available training data increases.

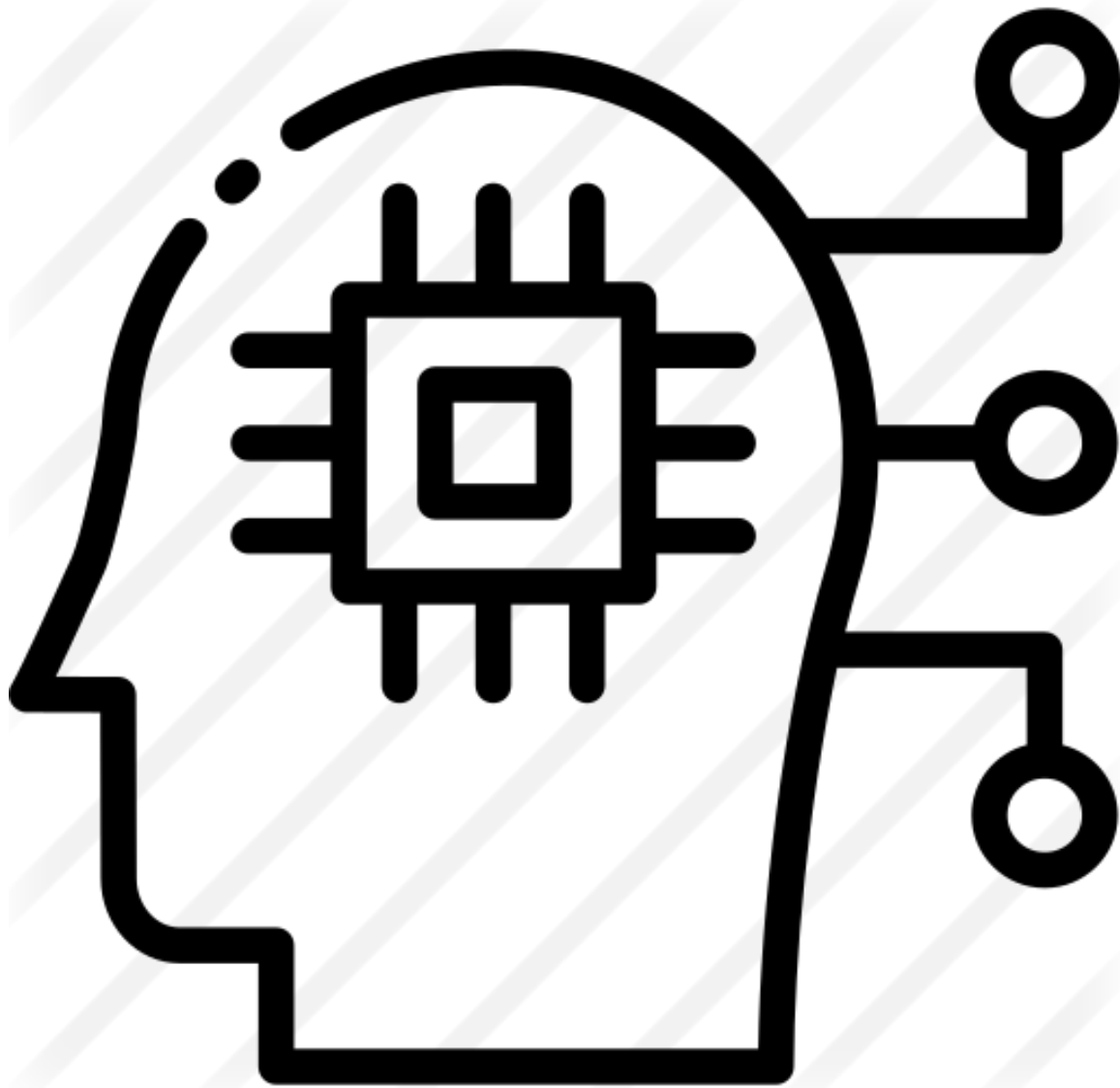
**More data is better!**

At some point, the learning curve flattens out, and more data does not contribute much!

# Comparing to a Baselines



- First step: get a **baseline**
  - The baseline is often a simple model.
  - Helps to determine how hard the task is.
  - Helps determine what a good accuracy is.
- **Weak baseline:** E.g., the most frequent label classifier
  - Gives all test instances the same label which is the most common label in the training set.
    - Example: For spam filtering, give every message the label “ham” since most messages are ham.
  - Accuracy might be very high if the problem is skewed (called class imbalance).
    - Example: If calling everything “ham” gets already 66% right, then a classifier that gets 68% isn’t a big improvement!
- **Strong baseline:** For research, we typically compare our model with the performance of previously published state-of-the-art methods.



# Types of Supervised ML Models

Regression: Predict a number

Classification: Predict a label

# Regression: Linear Regression

- Model:  $h_{\mathbf{w}}(\mathbf{x}_j) = w_0 + w_1x_{j,1} + \dots + w_nx_{j,n} = \sum_i w_ix_{j,i} = \mathbf{w}^T \mathbf{x}_j$
- Empirical loss:  $L(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$
- Gradient:  $\nabla L(\mathbf{w}) = 2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$
- Minimum loss:  $\nabla L(\mathbf{w}) = 0$

Squared error loss over the whole data matrix  $\mathbf{X}$

The gradient is a vector of partial derivatives

$$\nabla L(\mathbf{w}) = \left[ \frac{\partial L}{\partial w_1}(\mathbf{w}), \frac{\partial L}{\partial w_2}(\mathbf{w}), \dots, \frac{\partial L}{\partial w_n}(\mathbf{w}) \right]^T$$

- **Solution:** Gradient descent

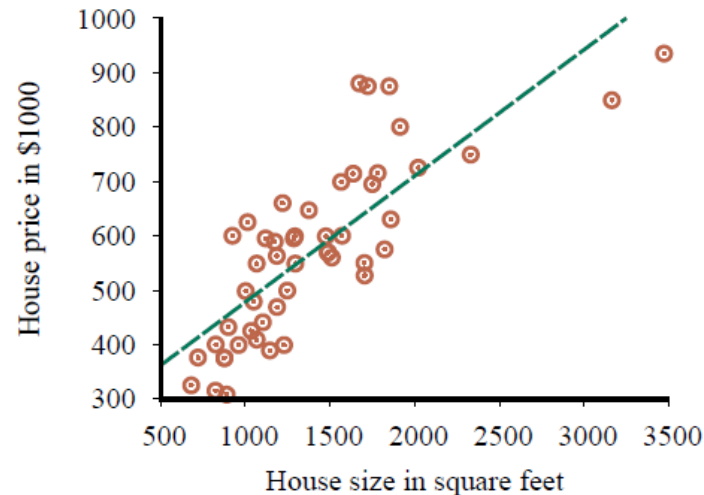
$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla L(\mathbf{w})$$

- **Alternative:** Analytical solution

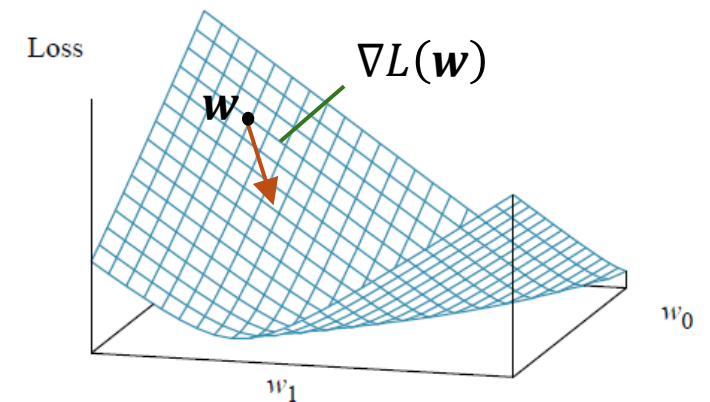
$$2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0} \quad (\text{solve for } \mathbf{w})$$

$\underbrace{\hspace{1.5cm}}$   
Pseudo inverse

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$



(a)



Convex loss function

(b)

$$h^*(\mathbf{x}) = \operatorname{argmax}_y P(Y = y \mid X = \mathbf{x})$$

# Naïve Bayes Classifier

- Approximates a Bayes classifier with the **naïve independence assumption** that all  $n$  features are conditional independent given the class.

$$h(\mathbf{x}) = \operatorname{argmax}_y P(y) \prod_{i=1}^n P(\mathbf{x}_i \mid y)$$

The priors  $P(y)$ s and the likelihoods  $P(\mathbf{x}_i \mid y)$ s are estimated from the data by (smoothed) counting.

- Gaussian Naïve Bayes Classifiers** extend the approach to **continuous features** by modeling the feature likelihood for each class  $y$  as a Gaussian probability density:

$$P(\mathbf{x}_i \mid y) \sim N(\mu_y, \sigma_y)$$

The parameters for the normal distribution  $N(\mu_y, \sigma_y)$  are estimated from data.

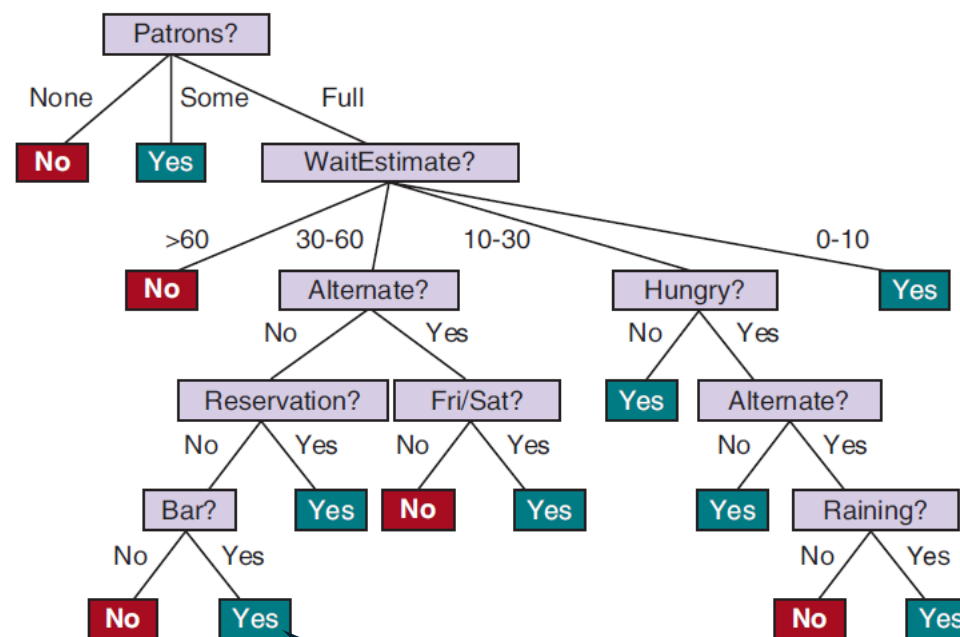
# Decision Trees

| Example         | Input Attributes |     |     |     |      |        |      |     |         |       | Output                |
|-----------------|------------------|-----|-----|-----|------|--------|------|-----|---------|-------|-----------------------|
|                 | Alt              | Bar | Fri | Hun | Pat  | Price  | Rain | Res | Type    | Est   | WillWait              |
| x <sub>1</sub>  | Yes              | No  | No  | Yes | Some | \$\$\$ | No   | Yes | French  | 0-10  | y <sub>1</sub> = Yes  |
| x <sub>2</sub>  | Yes              | No  | No  | Yes | Full | \$     | No   | No  | Thai    | 30-60 | y <sub>2</sub> = No   |
| x <sub>3</sub>  | No               | Yes | No  | No  | Some | \$     | No   | No  | Burger  | 0-10  | y <sub>3</sub> = Yes  |
| x <sub>4</sub>  | Yes              | No  | Yes | Yes | Full | \$     | Yes  | No  | Thai    | 10-30 | y <sub>4</sub> = Yes  |
| x <sub>5</sub>  | Yes              | No  | Yes | No  | Full | \$\$\$ | No   | Yes | French  | >60   | y <sub>5</sub> = No   |
| x <sub>6</sub>  | No               | Yes | No  | Yes | Some | \$\$   | Yes  | Yes | Italian | 0-10  | y <sub>6</sub> = Yes  |
| x <sub>7</sub>  | No               | Yes | No  | No  | None | \$     | Yes  | No  | Burger  | 0-10  | y <sub>7</sub> = No   |
| x <sub>8</sub>  | No               | No  | No  | Yes | Some | \$\$   | Yes  | Yes | Thai    | 0-10  | y <sub>8</sub> = Yes  |
| x <sub>9</sub>  | No               | Yes | Yes | No  | Full | \$     | Yes  | No  | Burger  | >60   | y <sub>9</sub> = No   |
| x <sub>10</sub> | Yes              | Yes | Yes | Yes | Full | \$\$\$ | No   | Yes | Italian | 10-30 | y <sub>10</sub> = No  |
| x <sub>11</sub> | No               | No  | No  | No  | None | \$     | No   | No  | Thai    | 0-10  | y <sub>11</sub> = No  |
| x <sub>12</sub> | Yes              | Yes | Yes | Yes | Full | \$     | No   | No  | Burger  | 30-60 | y <sub>12</sub> = Yes |

- A **sequence of decisions** represented as a tree.
- Many implementations are available that differ by
  - How to select features to split?
  - When to stop splitting?
  - Is the tree pruned?
- Approximates a Bayesian classifier by

$$h(\mathbf{x}) = \operatorname{argmax}_y P(Y = y \mid \text{leafNodeMatching}(\mathbf{x}))$$

$$\text{Bayes Classifier} \\ h^*(\mathbf{x}) = \operatorname{argmax}_y P(Y = y \mid X = \mathbf{x})$$

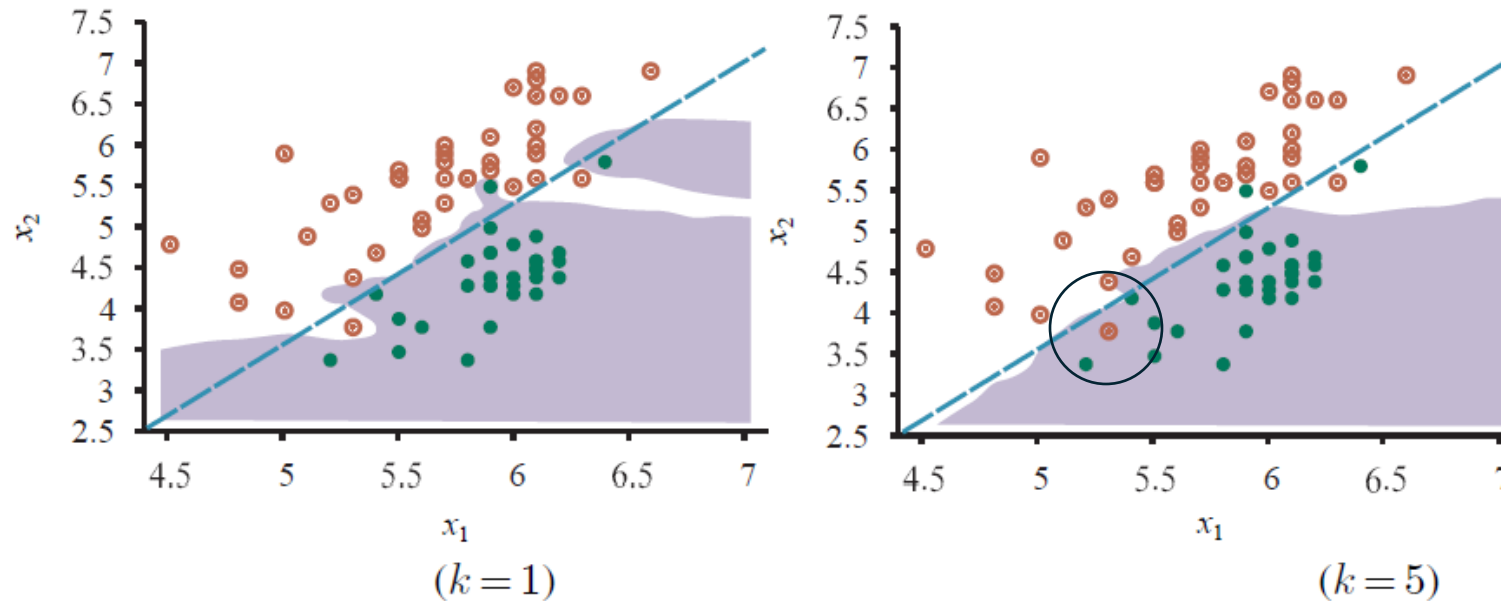


Class labels for leaf nodes are decided by the majority of the training data ending up in the leaf node. The probability can be estimated as:

$$\hat{P}(\text{Yes} \mid \text{node } N) = \frac{N_{\text{Yes}}}{N_{\text{Yes}} + N_{\text{No}}}$$

$$h^*(\mathbf{x}) = \underset{y}{\operatorname{argmax}} P(Y = y \mid X = \mathbf{x})$$

# K-Nearest Neighbors Classifier

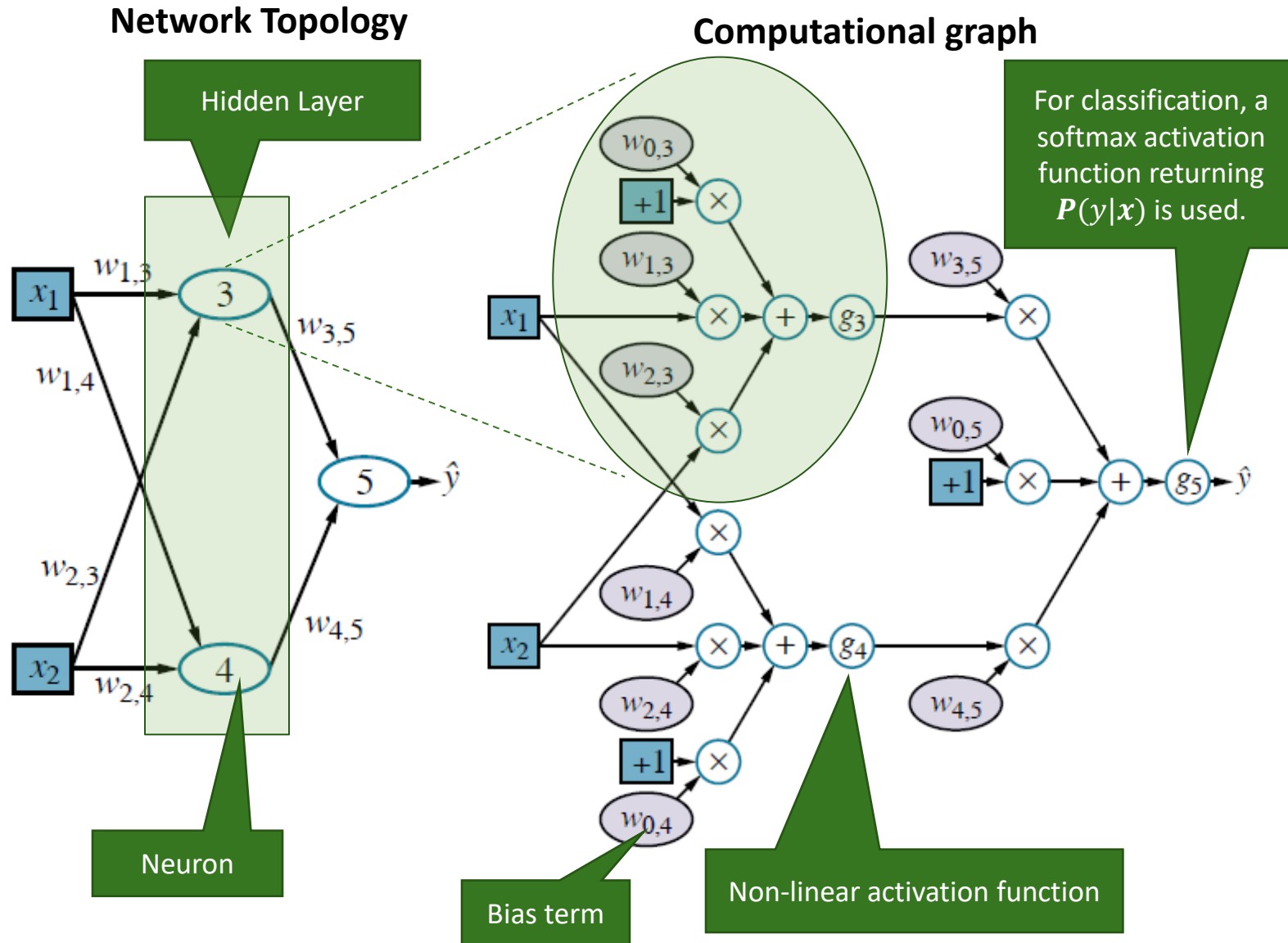


- Class is predicted by looking at the majority in the set of the  $k$  nearest **neighbors**.  $k$  is a hyperparameter. Larger  $k$  smooths the decision boundary.
- Neighbors are found using a distance measure (e.g., Euclidean distance between points).
- Approximates a Bayesian classifier by

$$h(\mathbf{x}) = \underset{y}{\operatorname{argmax}} P(Y = y \mid \text{neighborhood}(\mathbf{x}))$$

# Most Used in AI: Artificial Neural Networks

Superscript  $[n]$  means the layer. Layer weights are collected in a matrix.



- Represent  $\hat{y} = h(x) = g^{[2]} \left( W^{[2]} g^{[1]} (W^{[1]} x) \right)$  as a network of weighted sums with non-linear **activation functions**  $g(\cdot)$  (e.g., sigmoid, ReLU).
- Learn weight matrices  $W$  from examples using gradient descent with **backpropagation** of prediction errors  $L(\hat{y}, y)$ .
- ANNs are **universal approximators**. Large networks can approximate any function (has no bias). **Regularization** is typically needed to avoid overfitting.
- **End-to-end learning**: The hidden layer performs “automatic feature engineering”
- **Deep learning** adds more hidden layers and layer types (e.g., convolution layers) for more efficient learning and transfer learning.

# Typical Use of Supervised ML for Intelligent Agents

## Learn a Policy

- Classification: Directly learn the best action for each state from examples.

$$a = h(\text{state features})$$

- This model can also be used as a **playout policy** for Monte Carlo tree search with data from self-play.

## Learn Evaluation Functions

- Regression: Learn evaluation functions to estimate state utilities.

$$\text{eval} = h(\text{state features})$$

- Can learn a **heuristic** for heuristic alpha-beta search.
- For reinforcement learning we can learn action values  $q(\text{state}, \text{action})$ .

## Learn Perception and Actuation

- **Natural language processing:** Use deep learning / word embeddings / language models to understand concepts, translate between languages, or generate text.
- **Speech recognition:** Identify the most likely sequence of words.
- **Vision:** Object recognition in images/videos. Generate images/video.
- **Robotics:** Learn how to move safely.

## Compressing Tables

- Neural networks can be used as a compact representation of tables that do not fit in memory. E.g.,
  - Joint and conditional probability tables
  - State utility tables (i.e., an evaluation function)
  - Q-Value tables in reinforcement learning

**Bottom line:** Learning a function is often more effective than hard-coding it. However, we do not always know how it performs for rare and edge cases!